

Simulation framework for Table 1

J. Dedecker, M.-L. Taupin, V. Runge and A.-S. Tocquet

2026-07-01

Contents

1	Objective	2
2	Packages	2
3	Global parameters	2
4	Fast Toeplitz-Gaussian generator	3
5	One Table 1 data set	3
6	Shared metadata	5
7	Example data set	5
8	Old model-free screening test	6
9	One replication: data generation and screening	8
10	Example: one replication	8
11	Monte Carlo simulation for one Table 1 column	9
12	Generate Table 1	10
13	Generated Table 1	11
14	Selection frequencies for X1 to X18	12
15	Save compact results	12

1 Objective

This document implements the continuous, heteroscedastic Table 1 data-generating process with a faster simulator for the correlated Gaussian blocks.

The improvement replaces dense covariance-matrix construction and Cholesky factorization by the exact stationary Gaussian AR(1) recursion

$$X_j = \rho X_{j-1} + \sqrt{1 - \rho^2} \varepsilon_j, \quad \varepsilon_j \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma^2),$$

which gives

$$\text{Cov}(X_j, X_k) = \sigma^2 \rho^{|j-k|}.$$

Thus the simulated covariance is the same Toeplitz covariance as in Table 1, but the large 1982-variable noise block is generated much faster.

2 Packages

```
knitr::opts_chunk$set(echo = TRUE, eval = TRUE, cache.path = "table1_cache_old_test/")
library(parallel)
```

3 Global parameters

```
# Table 1 dimensions
p <- 2000L
N <- 500L

#set.seed(20260701)
base_seed <- 20260701L
replication_seeds <- base_seed + seq_len(N)

n_values <- c(500L, 1000L, 2000L, 2500L)

# Variable groups
n_active <- 10L
n_proxy_and_noise <- 8L
n_toeplitz_noise <- p - n_active - n_proxy_and_noise # 1982

# Toeplitz correlations
rho_active <- 0.57
rho_toeplitz_noise <- 0.70

# Regression coefficients, including the intercept
theta_star <- c(
  -1,
  3.619350,
  -3.274923,
```

```

    2.963273,
    -2.681280,
    2, 4, 6, 3, 2, 4
  )

stopifnot(length(theta_star) == n_active + 1L)
stopifnot(n_toeplitz_noise == 1982L)

```

4 Fast Toeplitz-Gaussian generator

```

simulate_toeplitz_gaussian_fast <- function(n, d, rho, sigma = 1)
{
  stopifnot(d >= 1L, abs(rho) < 1, sigma > 0)

  X <- matrix(0, nrow = n, ncol = d)

  # A stationary AR(1) process starts with marginal variance sigma^2.
  X[, 1L] <- rnorm(n, mean = 0, sd = sigma)

  if (d > 1L) {
    innovation_sd <- sigma * sqrt(1 - rho^2)

    # Draw all innovations at once to reduce random-number-generation overhead.
    innovations <- matrix(
      rnorm(n * (d - 1L), mean = 0, sd = innovation_sd),
      nrow = n,
      ncol = d - 1L
    )

    # Each row is a stationary Gaussian AR(1) sequence.
    for (j in 2:d) {
      X[, j] <- rho * X[, j - 1L] + innovations[, j - 1L]
    }
  }

  X
}

```

5 One Table 1 data set

This function returns only X and Y by default. It does not create column names or duplicate metadata at every replication, reducing memory allocation in Monte Carlo experiments.

```

simulate_table1_dataset_fast <- function(
  n,
  p = 2000,
  rho_active = 0.57,
  rho_toeplitz_noise = 0.70,
  theta_star = c(-1, 3.619350, -3.274923, 2.963273, -2.681280, 2, 4, 6, 3, 2, 4),

```

```

    add_names = FALSE
) {
  # -----
  # Group A: X1,...,X10
  # -----

  X1_to_X5 <- simulateToeplitzGaussianFast(
    n = n,
    d = 5L,
    rho = rho_active,
    sigma = 1
  )

  X6 <- rbinom(n, size = 1L, prob = 0.35)
  X7 <- rchisq(n, df = 2)
  X8 <- X1_to_X5[, 1L] * rpois(n, lambda = 2)
  X9 <- X1_to_X5[, 2L] * rnorm(n, mean = 1, sd = 1)
  X10 <- rt(n, df = 14)

  X_active <- cbind(X1_to_X5, X6, X7, X8, X9, X10)

  # -----
  # Response Y
  # -----

  linear_predictor <- drop(theta_star[1L] + X_active %*% theta_star[-1L])
  epsilon <- X_active[, 5L] * (rchisq(n, df = 3) - 3)
  Y <- abs(linear_predictor)^0.8 + epsilon / 3

  # -----
  # Groups B and C: X11,...,X18
  # -----

  Z3 <- rnorm(n, mean = 0, sd = 0.40)
  Z4 <- rnorm(n, mean = 0, sd = 0.02)
  Z5 <- rnorm(n, mean = 0, sd = 0.35)
  Z6 <- rnorm(n, mean = 0, sd = 0.55)

  X_proxy_and_noise <- cbind(
    X_active[, 1L] + Z3, # X11
    X_active[, 3L] + Z4, # X12
    X_active[, 4L] + Z5, # X13
    X_active[, 5L] + Z6, # X14
    Z3, Z4, Z5, Z6      # X15,...,X18
  )

  # -----
  # Group D: X19,...,X2000
  # -----

  X19_to_X2000 <- simulateToeplitzGaussianFast(
    n = n,
    d = p - 18L,

```

```

    rho = rho_toeplitz_noise,
    sigma = 1
  )

X <- cbind(X_active, X_proxy_and_noise, X19_to_X2000)

stopifnot(ncol(X) == p)
stopifnot(length(Y) == n)

if (add_names) {
  colnames(X) <- paste0("X", seq_len(p))
}

list(X = X, Y = Y)
}

```

6 Shared metadata

Keep this object once, outside Monte Carlo replications.

```

table1_metadata <- list(
  p = p,
  N = N,
  n_values = n_values,
  theta_star = theta_star,
  rho_active = rho_active,
  rho_toeplitz_noise = rho_toeplitz_noise,
  active_indices = 1:10,
  proxy_indices = 11:14,
  null_noise_indices = 15:18,
  toeplitz_noise_indices = 19:p,
  table1_positive_indices = 1:14,
  table1_null_indices = 15:p
)

```

7 Example data set

```

example_dataset <- simulate_table1_dataset_fast(
  n = 500L,
  add_names = FALSE
)

dim(example_dataset$X)

```

```
## [1] 500 2000
```

```
head(example_dataset$X[, 1:12])
```

```
##
## [1,] -0.9392383  0.07347083  0.9385957 -0.1178735 -0.29155306  0 0.3522528
## [2,]  0.1056816 -0.70589863 -0.6090986  0.6517686  1.55986633  0 1.5930120
## [3,]  1.3762177  0.44093908  0.9252364  1.7165673  1.29661965  0 1.1587221
## [4,]  0.7279375  1.52425511  0.7766027  0.1245216 -0.02274846  0 1.8357960
## [5,] -1.3350019 -0.83108541 -1.5845054  0.6296702 -0.05742767  0 0.5671875
## [6,]  3.2791770  1.67138940  0.1312547  0.3457485  1.24204511  1 0.1254116
##
##      X8      X9      X10
## [1,] -0.9392383  0.1404058 -0.1383543 -0.8214480  0.9387534
## [2,]  0.2113631 -0.3273861  0.9689251  0.6262978 -0.6082170
## [3,]  4.1286532  0.5331813 -1.1375238  0.7188378  0.8941578
## [4,]  2.1838124 -1.0559328 -0.7657753  0.8813079  0.7802752
## [5,] -4.0050058  0.3140670 -0.5895165 -1.6474121 -1.5891035
## [6,]  6.5583540 -0.2643063  1.1780934  3.3743489  0.1388230
```

```
head(example_dataset$Y)
```

```
## [1]  2.581287  8.779705  9.379426  6.792348 10.879173 16.655272
```

```
# Empirical correlation of X1,...,X5, to inspect the rho = 0.57 block:
round(cor(example_dataset$X[, 1:5]), 3)
```

```
##
##  1.000 0.567 0.342 0.166 0.087
##  0.567 1.000 0.568 0.286 0.148
##  0.342 0.568 1.000 0.499 0.263
##  0.166 0.286 0.499 1.000 0.586
##  0.087 0.148 0.263 0.586 1.000
```

```
# Empirical correlation of X150,...,X155, to inspect the rho = 0.7 block:
round(cor(example_dataset$X[, 150:155]), 3)
```

```
##
##  1.000 0.707 0.489 0.358 0.227 0.125
##  0.707 1.000 0.701 0.504 0.340 0.190
##  0.489 0.701 1.000 0.721 0.486 0.326
##  0.358 0.504 0.721 1.000 0.695 0.460
##  0.227 0.340 0.486 0.695 1.000 0.660
##  0.125 0.190 0.326 0.460 0.660 1.000
```

8 Old model-free screening test

The following function implements the original marginal screening test directly from the model-free screening procedure. It accepts the simulated design matrix X and response vector Y , computes the marginal slopes, White variance estimates, studentized statistics, and selected variables.

```
screening_test_ML <- function(X, Y, q_n = 0.10) {
  X <- as.matrix(X)
  Y <- as.numeric(Y)
```

```

n <- nrow(X)
p <- ncol(X)

if (length(Y) != n) {
  stop("Y must have the same number of observations as rows of X.")
}
if (!is.numeric(q_n) || length(q_n) != 1L || q_n <= 0 || q_n >= 1) {
  stop("q_n must be one number strictly between 0 and 1.")
}

x_means <- colMeans(X)
y_mean <- mean(Y)

X_centered <- sweep(X, 2L, x_means, FUN = "-")
Y_centered <- Y - y_mean

# Marginal slope estimates: tau_hat(j).
sxx <- colSums(X_centered^2)
if (any(sxx <= 0)) {
  stop("Every column of X must have strictly positive sample variance.")
}

tau_hat <- colSums(X_centered * Y_centered) / sxx
b_hat <- y_mean - tau_hat * x_means

# One residual column for each marginal regression:
# residual[i, j] = Y[i] - b_hat[j] - X[i, j] * tau_hat[j].
fitted_values <- sweep(X, 2L, tau_hat, FUN = "*")
fitted_values <- sweep(fitted_values, 2L, b_hat, FUN = "+")
residuals_hat <- sweep(-fitted_values, 1L, Y, FUN = "+")

# White estimator:
# V_hat(j) = { n^{-1} sum X_centered(i, j)^2 residual(i, j)^2 } /
#           { n^{-1} sum X_centered(i, j)^2 }^2.
var_x_n <- sxx / n
V_hat <- colMeans(X_centered^2 * residuals_hat^2) / var_x_n^2

gamma_hat <- sqrt(n) * tau_hat / sqrt(V_hat)
threshold_gamma <- qnorm(1 - q_n / 2)

selected <- abs(gamma_hat) >= threshold_gamma

list(
  scores = gamma_hat,
  selected = selected,
  selected_indices = which(selected),
  threshold = threshold_gamma,
  tau_hat = tau_hat,
  white_variance = V_hat
)
}

```

9 One replication: data generation and screening

This function generates one Table 1 data set, immediately applies the old model-free screening test, and returns only the compact outputs needed for the Monte Carlo summaries.

```
run_one_table1_replication <- function(n, q = 0.10) {
  dat <- simulate_table1_dataset_fast(
    n = n,
    p = p,
    rho_active = rho_active,
    rho_toeplitz_noise = rho_toeplitz_noise,
    theta_star = theta_star,
    add_names = FALSE
  )

  test_result <- screening_test_ML(
    X = dat$X,
    Y = dat$Y,
    q_n = q
  )

  selected <- as.logical(test_result$selected)

  list(
    selected_first18 = as.integer(selected[1:18]),
    selected_positive = sum(selected[1:14]),
    selected_null = sum(selected[15:p]),
    selected_total = sum(selected),
    threshold = test_result$threshold
  )
}
```

10 Example: one replication

```
one_result <- run_one_table1_replication(n = 500L, q = 0.10)

one_replication_summary <- data.frame(
  n = 500L,
  p = p,
  q = 0.10,
  threshold_gamma = one_result$threshold,
  selected_among_X1_to_X14 = one_result$selected_positive,
  selected_among_X15_to_X2000 = one_result$selected_null,
  selected_model_size = one_result$selected_total,
  TPR_contribution = one_result$selected_positive / 14,
  FPR_contribution = one_result$selected_null / (p - 14)
)

knitr::kable(
  one_replication_summary,
  digits = 4,
```



```
caption = "One replication: quantities that are averaged for Table 1"
)
```

Table 1: One replication: quantities that are averaged for Table 1

n	p	q	threshold_g	selected_among_X	selected_X14	selected_X15	selected_X2000	ITDe	EBDe	contribution
500	2000	0.1	1.6449	13	240	253	0.9286	0.1208		

```
knitr::kable(
  data.frame(
    variable = paste0("X", 1:18),
    selected = one_result$selected_first18
  ),
  caption = "One replication: selection indicators for X1 to X18"
)
```

Table 2: One replication: selection indicators for X1 to X18

variable	selected
X1	1
X2	1
X3	1
X4	1
X5	1
X6	1
X7	1
X8	1
X9	1
X10	1
X11	1
X12	1
X13	0
X14	1
X15	0
X16	0
X17	0
X18	0

11 Monte Carlo simulation for one Table 1 column

The function below runs N replications for a fixed sample size. It stores only compact screening outputs, not the large X and Y objects.

```
run_table1_column <- function(n, N = N, q = 0.10,
                              mc.cores = 1L,
                              base_seed = 20260701L)
{
```

```

replication_seeds <- base_seed + seq_len(N)

one_draw <- function(i) {
  set.seed(replication_seeds[i])
  run_one_table1_replication(n = n, q = q)
}

if (.Platform$OS.type == "windows" || mc.cores <= 1L) {
  repetitions <- lapply(seq_len(N), one_draw)
} else {
  repetitions <- parallel::mclapply(
    X = seq_len(N),
    FUN = one_draw,
    mc.cores = mc.cores,
    mc.set.seed = FALSE,
    mc.preschedule = FALSE
  )
}

selection_first18 <- do.call(
  rbind,
  lapply(repetitions, `[`, "selected_first18")
)

selected_positive <- vapply(
  repetitions, `[`, numeric(1), "selected_positive"
)
selected_null <- vapply(
  repetitions, `[`, numeric(1), "selected_null"
)
selected_total <- vapply(
  repetitions, `[`, numeric(1), "selected_total"
)

list(
  n = n,
  N = N,
  q = q,
  threshold = repetitions[[1]]$threshold,
  selection_frequency_first18 = colMeans(selection_first18),
  TPR = mean(selected_positive / 14),
  FPR = mean(selected_null / (p - 14)),
  mean_selected_model_size = mean(selected_total)
)
}

```

12 Generate Table 1

This chunk runs the simulation for $n = 500, 1000, 2000, 2500$, with $N = 500$ Monte Carlo replications per sample size. For a quick test, temporarily set `N <- 5L` in the parameter section.

Table 3: Generated Table 1 using the original model-free screening test ($N = 500$, $p = 2000$, $q = 0.10$)

	n	TPR	FPR	Mean_selected_model_size
n=500	500	0.816	0.103	216.722
n=1000	1000	0.899	0.102	214.478
n=2000	2000	0.961	0.101	213.930
n=2500	2500	0.977	0.101	214.250

```
n_cores <- min(8L, parallel::detectCores(logical = FALSE))

table1_runs <- lapply(
  n_values,
  run_table1_column,
  N = N,
  q = 0.10,
  mc.cores = n_cores
)

names(table1_runs) <- paste0("n=", n_values)
```

13 Generated Table 1

```
table1_summary <- data.frame(
  n = n_values,
  TPR = vapply(table1_runs, `[`, numeric(1), "TPR"),
  FPR = vapply(table1_runs, `[`, numeric(1), "FPR"),
  Mean_selected_model_size = vapply(
    table1_runs, `[`, numeric(1), "mean_selected_model_size"
  )
)

knitr::kable(
  table1_summary,
  format = "latex",
  booktabs = TRUE,
  digits = 3,
  align = c("c", "c", "c", "c"),
  caption = paste0(
    "Generated Table 1 using the original model-free screening test ",
    "(N = ", N, ", p = ", p, ", q = 0.10)"
  )
)
```

Table 4: Monte Carlo selection frequencies for X1 to X18

variable	n=500	n=1000	n=2000	n=2500
X1	1.000	1.000	1.000	1.000
X2	0.998	1.000	1.000	1.000
X3	0.994	1.000	1.000	1.000
X4	0.372	0.524	0.766	0.856
X5	0.622	0.882	0.980	1.000
X6	0.598	0.860	0.988	0.996
X7	1.000	1.000	1.000	1.000
X8	0.994	1.000	1.000	1.000
X9	0.998	1.000	1.000	1.000
X10	0.990	1.000	1.000	1.000
X11	1.000	1.000	1.000	1.000
X12	0.992	1.000	1.000	1.000
X13	0.350	0.526	0.752	0.840
X14	0.518	0.800	0.968	0.990
X15	0.114	0.090	0.102	0.078
X16	0.108	0.086	0.114	0.118
X17	0.106	0.088	0.108	0.118
X18	0.104	0.108	0.114	0.118

14 Selection frequencies for X1 to X18

```
selection_first18_table <- data.frame(
  variable = paste0("X", 1:18),
  do.call(
    cbind,
    lapply(table1_runs, function(x) x$selection_frequency_first18)
  ),
  check.names = FALSE
)

colnames(selection_first18_table)[-1] <- names(table1_runs)

knitr::kable(
  selection_first18_table,
  format = "latex",
  booktabs = TRUE,
  digits = 3,
  caption = "Monte Carlo selection frequencies for X1 to X18"
)
```

15 Save compact results

```
saveRDS(  
  object = list(  
    metadata = table1_metadata,  
    table1_summary = table1_summary,  
    table1_runs = table1_runs  
  ),  
  file = "table1_old_screening_results.rds"  
)  
  
write.csv(  
  table1_summary,  
  file = "table1_old_screening_summary.csv",  
  row.names = FALSE  
)
```